INDEX

NAME: Jaisurya.R STD.: CSE SEC.: ROLL NO.: 240701204. SUB.: CN Observation.

S. No.	Date	Title	Page No.	Teacher's Sign / Remarks
1.	27/7/26	Study of various Network Commands used in Linux and Windows		
2.	28/7/26	Study of Network Cables		
3.	3/8/26	Experiments on CISCO PACKET TRACER.		
4.				
4.	10/8/26	Setup and configure LAN using Switch and Ethernet Cables.		
5.	17/8/26	Experiments on Packet Capture tool: Wireshark.		
6.	24/8/26	Error Correction at Data Link Layer.		
7.	7/9/26	Flow Control at Data Link Layer.		
8.	21/9/26	NMAP to Discover Live Hosts Using Nmap Scans (ARP, ICMP, TCP/UDP) on the Try Hackme Platform		

S. No.	Date	Title	Page No.	Teacher's Sign / Remarks
9.	28/9/26	Implementation of Subnetting in CISCO Packet Tracer Simulator.		
10.	29/9/26	Internetworking with routers in CISCO Packet Tracer Simulator.		
11.	5/10/26	Routing at Network Layer.		
12.	6/10/26	End - End communication at Transport Layer.		
13.	6/10/26	Implement your own ping program.		
14.	12/10/26	Write a code using using RAW Sockets to implement packet Sniffing		
~~15.~~				

Completed 19/10/26

EXP NO: 1

Study of Various Network Comm used in Linux and Windows

AIM:

To Study of various Network Commands used in Linux and Windows.

Basic Networking Commands.

1. arpa : ARP is short form of address resolution protocol. It will show the IP address of your computer along with the IP address and MAC address of your router.

o/p:

gateway (10.0.2.2) at 52:33:0a:00:02:02 [ether] on enp0s3

2. hostname : This is the simplest of all TCP/IP commands. It simply display the name of your computer.

o/p:

fedora

3. ipconfig/all: This command displays detailed configuration information about your TCP/IP connection including Router, Gateway, DNS, DHCP, and type of Ethernet adapter in your System.

4. net stat:

netstat displays a variety of statistics about a computers archive TCP/IP connections.

o/p:

unix 3[ ] STREAM CONNECTED 12426.

5. ifconfig:

This command displays detailed config information about your TCP/IP connection including router, gateway, DNS, DHCP & type of ethernet adapter in your system.

o/p:

enp0s flags = 4163<UP, BROADCAST, RUNNING, MULTICAST mtu 1500

inet 10.0.2.15 netmask 255.255.255.0

broadcast.

6. nslookup:

It is a tool used to perform DNS lookups in Linux. It is used to display such as the IP address of particular computer.

eg: nslookup www.google.com.

o/p: Server: 127.00.52

Address: 127.0.0.53 #53

non-authoritative answer

name: www.google.com

Address: 142.251.152.119.

RESULT:

Thus, the various commands in Linux a
windows are executed.

NO:2.

STUDY OF NETWORK CABLES

Aim:

Study of different types of Network Cables.

a) Understand different types of network cable.

1. Unshielded Twisted Pair (UTP) Cable
2. Shielded Twisted Pair (STP) Cable.
3. Coaxial Cable.
4. Fibre Optic Cable.

Cable type	Category	Maximum Data Transmission	Advantages / Disadvantages	Application / Use
UTP	Category 3	10 bps	Advantages. * cheaper in cost * Easy to install as they have a smaller diameter. Disadvantages. More prone to (EMI) Electro magnetic interference and noise.	10 Base -T Ethernet Fast Ethernet, Gigabit Ethernet
	Category 5	Upto 100 Mbps.		
	Category 5e	~~10~~ 1 Gbps.		
STP	Category 6, 6a	10 Gbps	Advantages. • Shielded • Faster than UTP. • Less susceptible to noise and interference. Disadvantages. • Expensive • Greater installation effort.	Gigabit Ethernet, 10 G Ethernet (55m) Widely used in data centres.
	Category 7	10 Gbps.		Gigabit Ethernet, 10 G Ethernet (100m)

Coaxial Cable.	RG-6 RG-59 RG-11	10-100 Mbps.	• High bandwidth • Immune to interference • Low Loss bandwidth • Versatile. Disadvantages • Limited distance • Cost • Size is bulky	Speed of signal is 500m, Television network High Speed internet Connections.
Fibre Optics Cable.	Single mode multi mode	100Gbps.	Advantages • High Speed • High bandwidth • High security • Long distance. Disadvantages. • Expensive • Requires Skilled installers.	Maximum distance of fibre optics Cable is arrond 100meters.

b) Make your Own Ethernet Cross-Over Cable / Straigt Cable.

Step1: To Start Construction of the device, begin by threading Shields onto the Cable.

Step2: Next, Strip approximately 1.5cm of Cable Shielding from both ends. The crimping tool has a round area to complete this task.

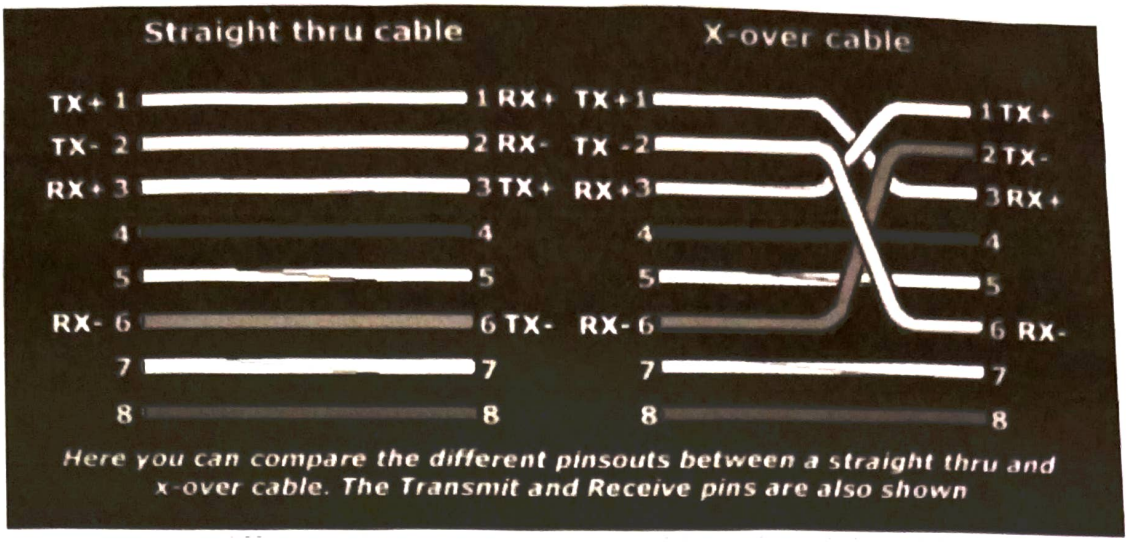
Step 3 : After you will need to untangle ... there Should be four "twisted pairs" Reference back to the sheet, arrange them from top to bottom. One end should be in arrangement A and the other in B

Step 4: Once the order is correct, bunch them together in a line, and if there are any stick out farther than others. Snip them back to create an even level. The difficult aspect is placing these into the RJ45 plug without messing up the order. To do so, hold the plug with the clip side facing away from you and have the gold pins facing toward you.

Step 5: Next, push the cable right in. The notch at the end of the plug needs to be just over the cable shielding, and if isn't that you stripped off too much shielding - simply snip the cables back a little more.

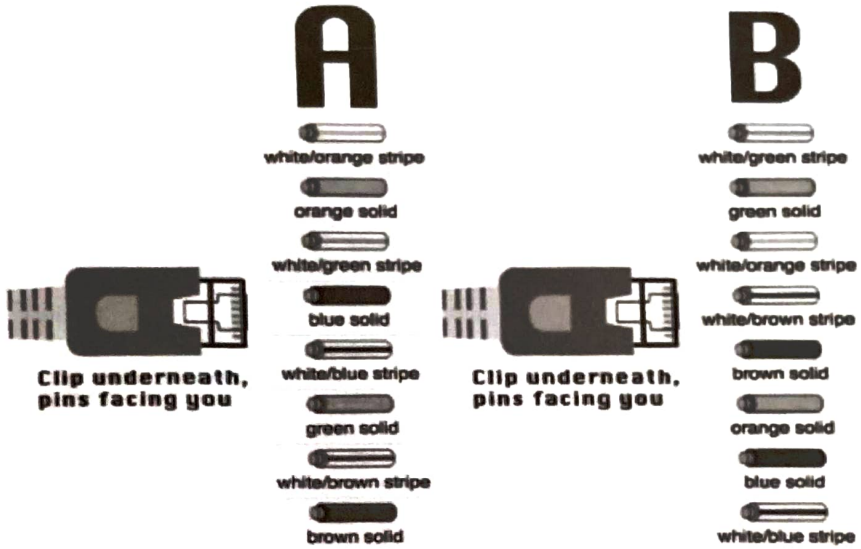
Step 6: After the wires are securely sitting inside the plug insert it into the crimping tool and push down.

Step 7: Lastly, repeat for the other end using diagram B, using diagram A.



Here you can compare the different pinsouts between a straight thru and x-over cable. The Transmit and Receive pins are also shown

Straight through network cable: both sides should be A
Crossover cable: One side A, one side B



EXP: 03.

EXPERIMENTS ON CISCO PACKET TRACER (Simulation Tool):

Aim:

To understand environment of CISCO Packet Tracer to design simple network.

INTRODUCTION:

A simulator, as the name suggest, simulates network devices and its environment. Packet Tracer is an exciting network design, simulation and modelling to

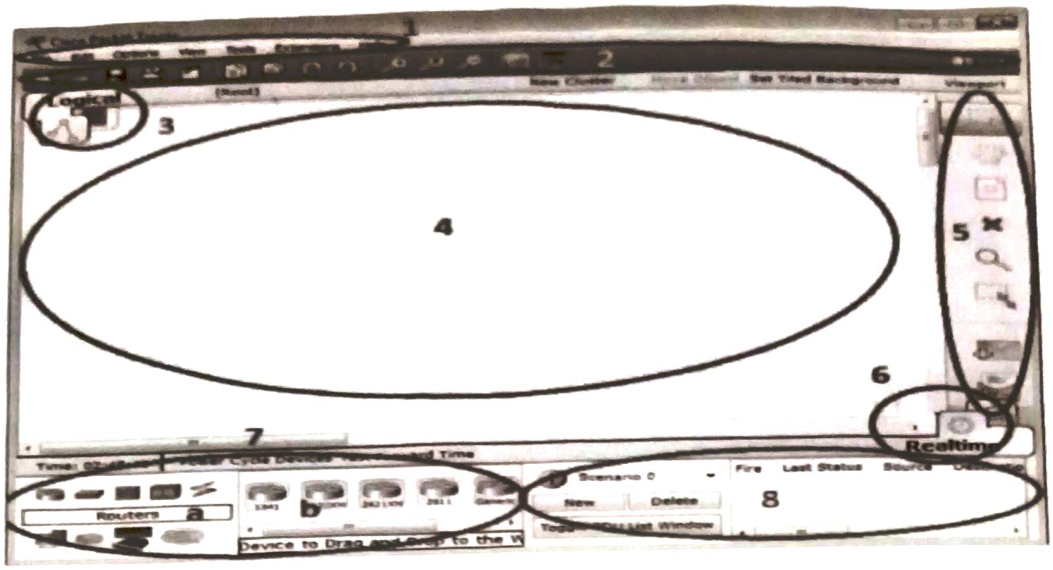
1. It allows you to model complex systems without the need for dedicated equipment.

2. It helps you to practice your network Configuration and troubleshoot skills via computer or a Android or iOS based mobile device.

3. It is available for both the Linux and Windows desktop environment.

USER INTERFACE OVERVIEW:

The layout of Packet Tracer is divided into several components. The components of the Packet Tracer interface are as follows: match the numbering with explanations.



1. Menu bar - This is a common menu found in all software applications; it is used to open, save, print, change preferences, and so on.

2. Main toolbar - This bar provides shortcut icons to menu options that are commonly accessed, such as open, save, Zoom, undo, and redo, and on the right-hand side.

3. Logical/Physical workspace tabs - These tabs allow you to toggle between the Logical and Physical work areas.

4. Workspace - This is the area where topologies are created and simulations are displayed.

5. Real-time Simulation tabs - these tabs are used to toggle between the real and Simulation modes. Buttons are also provided to control the time, and to capture the packets.

7. Network Component box - This Component Contains all the network and end devices available with Packet Tracer, and is further divided into two areas: Area & Devices - type selection box.

8. User - created packet box - Users can create highly Customized packets to test their topology from this area

d) Analyse the behaviour of network devices using CISCO PACKET TRACER Simulator.

1. From the network component box, click and drag-n-drop the below components:

a. 4 Generic PCs and One HUB.

b. 4 Generic PCs and One Switch.

Double Tap to Edit

2 Click on Connections:

a. Click on Copper straight - Through cable,

b. Select one of the PC and Connect it to HUB using the cable. The link LED should glow in green, indicating that the link is up. Similarly Connect remaining 3 PCs to the HUB.

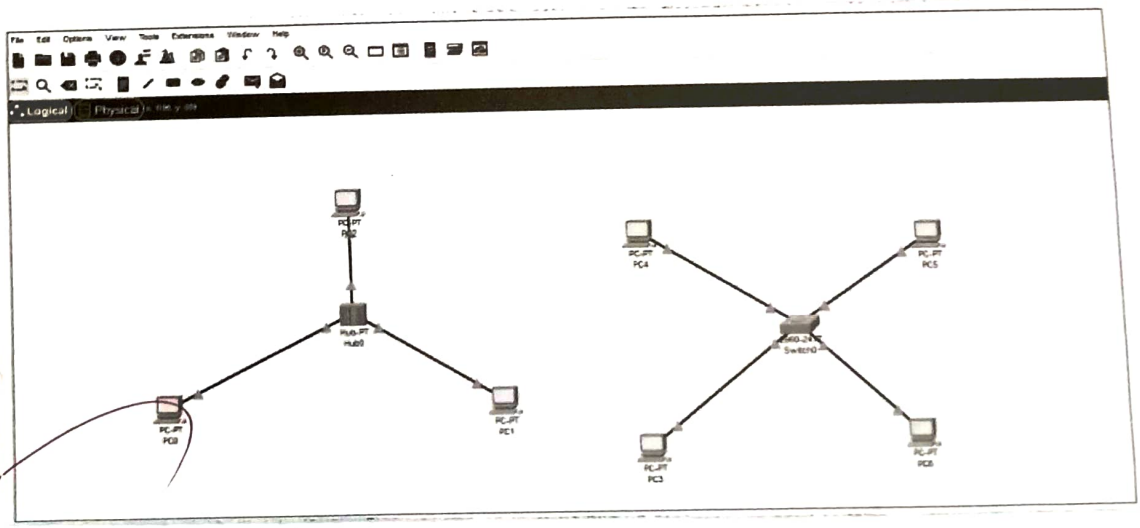
c. Similarly Connect 4 PCs to the Switch with Copper Straight - through cable.

3. Click on the PCs connected to hub, go to the Desktop tab, Click on IP Configuration and enter an IP address and Subnet mask. Here the default gateway and DNS Server information is not needed as there are only two end devices in the network.

4. Observe the flow of PDU from source PC to destination PC by selecting the Realtime mode of Simulation.

5. Repeat Step # 3 to Step # 5 for the PCs Connected to the Switch.

6. Observe how HUB and Switch are forwarding the PDU and write your observation and Conclusion about the behaviours of Switch and Hu



EXP. No. 4. Setup and Configure a LAN using Switch and Ethernet Cables in your Lab.

AIM:

Setup and Configure a LAN using a Switch and Ethernet Cables in your Lab.

What is LAN?

A LAN refers to a network that connects devices within a limited area, such as an office building, school, or home. It enables user to share resources, including data, printers, and internet access. LAN connects devices to promote collaboration and transfer information between users, such as Computers, printers, servers, and Switches.

How to set up a LAN.

Step 1: Plan and Design an appropriate network topology taking into account network requirements and equipment location.

Step 2: You can take 2 computers, a switch with 8, 16 and 24 ports which is sufficient for network of these sizes, and 4 Ethernet Cables.

Step3:

Connect your Computers to network Switch via an Ethernet Cable, which is as simple as plugging one end of the Ethernet Cable into your computer and the other end into your network switch.

Step4: Assign IP address to your PCs

1. Log on to the client Computer as Administrator or as Owner.

2. Click Network and Internet Connects.

3. Right click Local Area Connection / Ethernet -> Go to Properties -> Select Internet Protocol (TCP IPV4) -> Click on Properties -> Select use the following IP address option as assign ip address.

Similarly assign IP address to all the PCs connected to switch. PC1 -IP address : 10.1.1.1. Subnet mask 255.0.0.0

PC2 -IP address - 10.1.1.2, Subnet mask 255.0.0.0 PC3 -IP address 10.1.1.3. Subnet mask 255.0.0.0.

PC4 -IP address 10.1.1.4. Subnet mask 255.0.0.0.

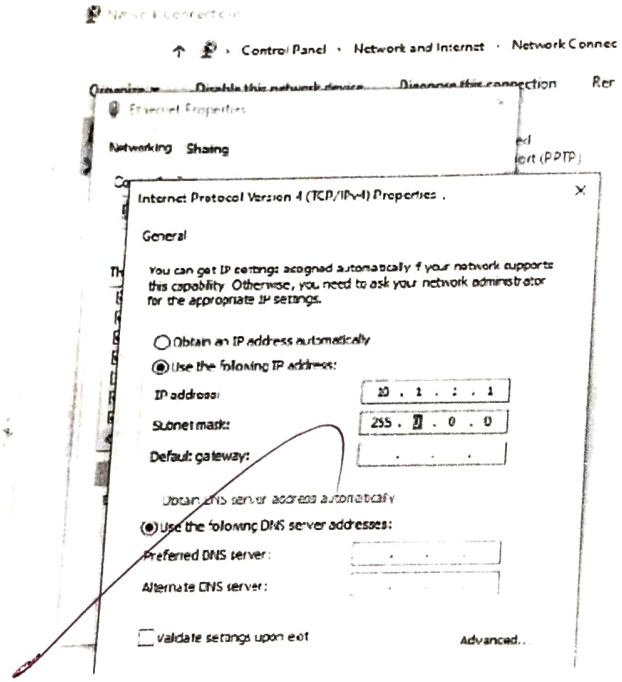
Step 5: Configure a network Switch:

1. Connect your computer to the switch. To access the switch's web interface, you will need to connect your computer to the switch using an Ethernet cable.

2. Log in to the web interface.

3. Configure basic settings: once you're logged in, you will be able to configure basic settings for the switch.

4. Assign IP address as: 10.1.1.5, Subnet mask 255.0.0.0.



Step 6:
check the connectivity between switch and other machine by using ping command in the command prompt of the device.

step 7: Select a folder → go to properties → click sharing tab → share it with everyone on the same LAN.

Step 8: Try to access the shared folder from others computers of the network.

EXP.NO:5 EXPERIMENTS ON PACKET CAPTURE TOOL: WIRESHARK

Aim:

Experiments on Packet Capture tool : Wireshark

Packet Sniffer:

* Sniffs messages being sent/received from/by your computer.
* Store and display the contents of the various protocol fields in the messages
* Passive program.
  - never sends packets itself
  - no packets addressed to it.
  - receives a copy of all packets (sent/received)

Packet Sniffer structure Diagnostic Tools

* Tcpdump.
  - Eg tcpdump -en -x host 10.129.41.2 -w exe 3.out.
* Wireshark
  - wireshark -r exe3.out.

1. Create a Filter to display only TCP/UDP Packets, Inspect the packets and provide the flow graph.

Procedure:

→ Select Local Area Connection in Wireshark.

→ Go to Capture option.

→ Select stop Capture automatically after 100 packets.

→ Then click start Capture

→ Search TCP packets in Search bar.

→ Save the packets.

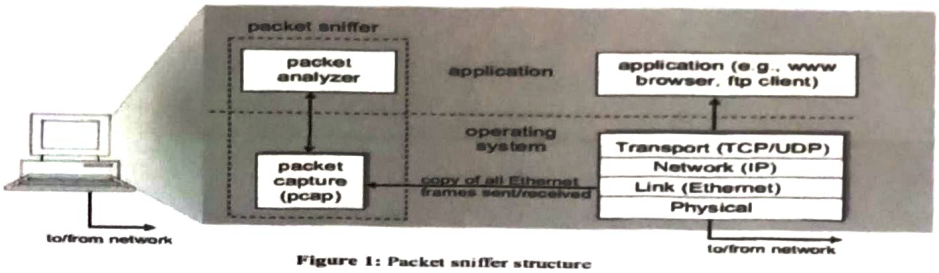


Figure 1: Packet sniffer structure

2. Create a Filter to display only ARP packets and inspect the packets. Procedure

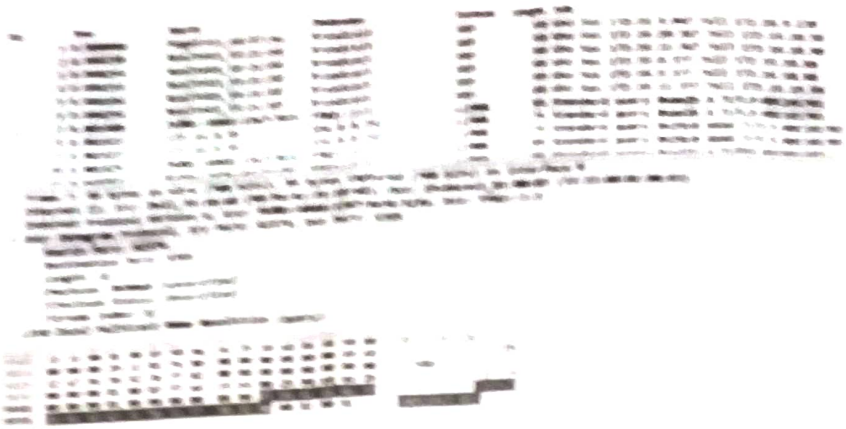
→ Go to Capture option.

→ Select stop Capture automatically after 100 packets.

→ Then click start Capture.

→ Search ARP packets in Search bar

→ Save the packets



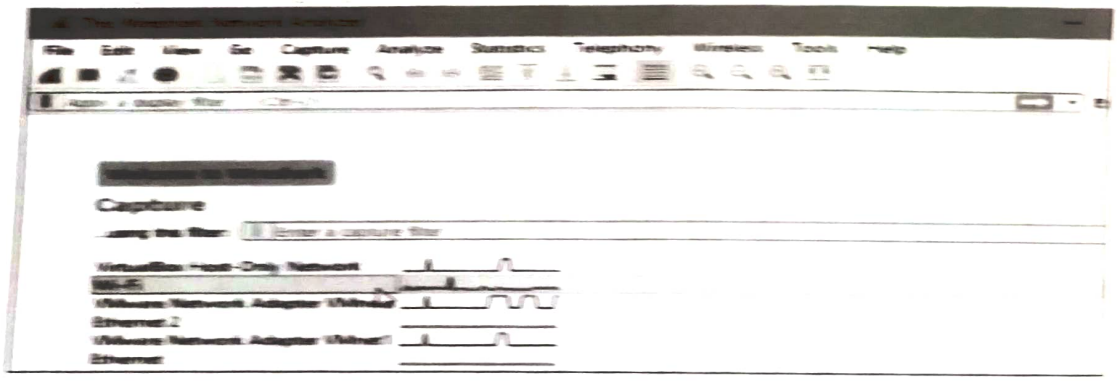
3. Create a filter to display only DNS packets and provide the flow graph. Include

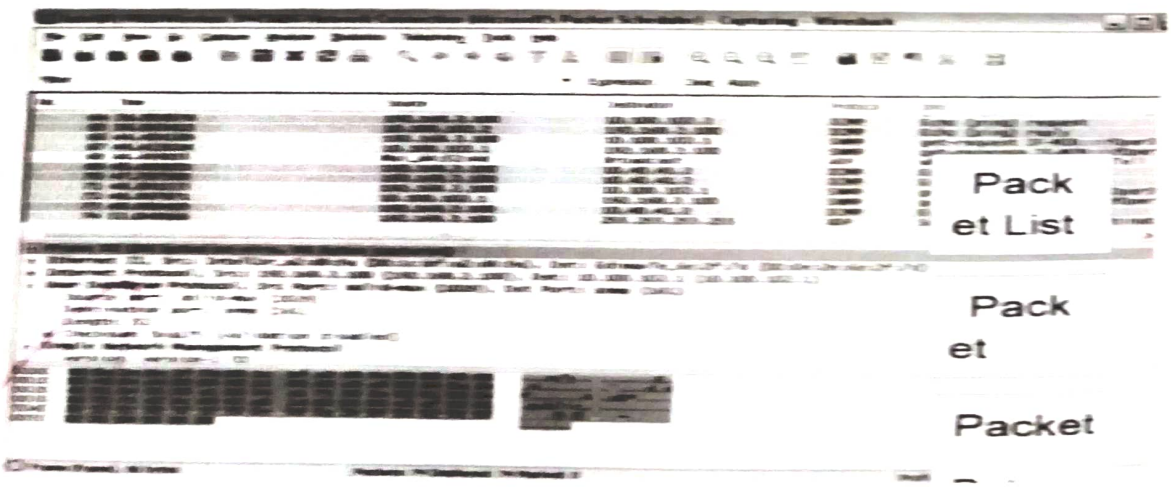
→ Go to Capture option

→ Select stop Capture automatically after 100 packets.

→ Then click Start Capture

→ Save the packets





4. Create a Filter to display only HTTP packets and inspect the packets Procedure.

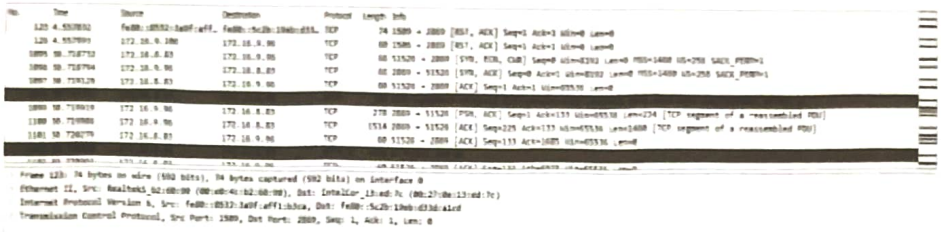
→ Select Local Area Connection in Wireshark

→ Go to Capture option.

→ Select Stop Capture automatically after 100 packets.

→ Search HTTP packets in search bar.

→ Save the packets.





5. Create a Filter to display only IP / ICMP packets and inspect the packets. Procedure.

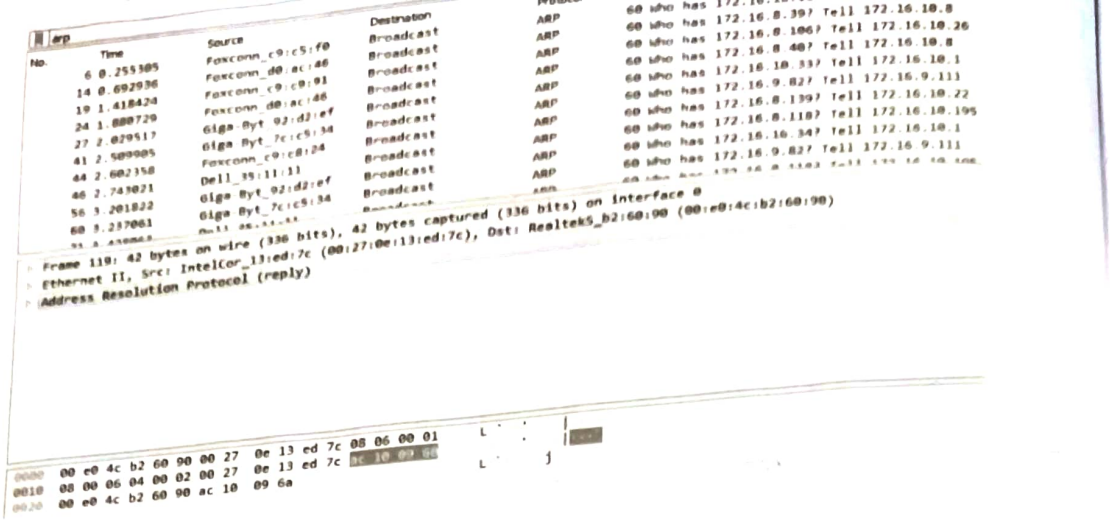
→ Select Local Area Connection in Wireshark

→ Go to Capture option.

→ ~~Select~~ Stop Capture automatically after 100 packets.

→ Then Click Start Capture.

→ Search ICMP / IP Packets in search bar.



6. Create a Filter to display only DHCP packets and inspect the packets. Procedure.

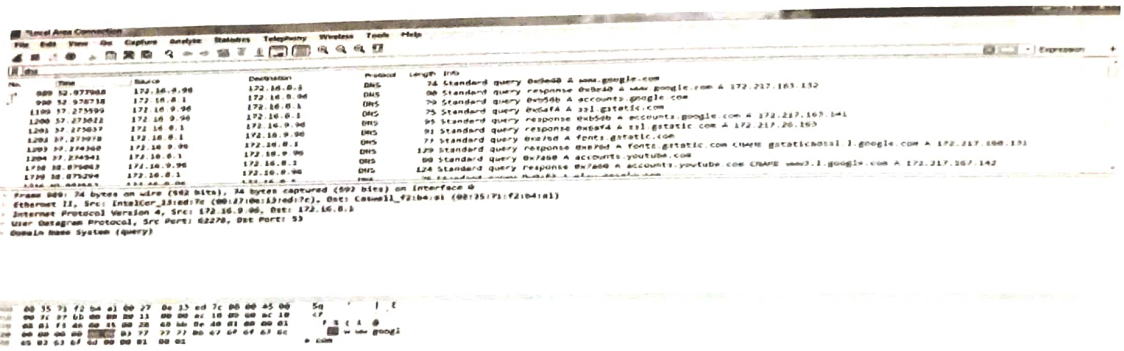
→ Go to Capture option.

→ Seled Stop Capture automatically after 100 pa

→ Then click Start Capture.

→ Search DHCP packets in search bar.

→ Save the packets.



EX.NO:6 ERROR CORRECTION AT DATA LINK LAYER:

Aim:

Write a program to implement error detection and correction using HAMMING Code Concept. Make a test run to input data stream and verify error correction feature.

Error Correction at Data Link Layer.

Hamming code is a set of error-correction codes that can be used to detect and correct the error that can occure when the data is transmitted from the sender to the receiver. It is a technique developed by R.W. Hamming for error correction.

Create sender program with below features

1. Input to sender file should be a text of any length. Program should convert the text to binary.

2. Apply hamming code concept on the binary data and add redundant bits to it.

3. Save this output in a file called channel.

Create a receiver program with below features.

1. Receiver program should read the input from channel file.

2. Apply hamming code on the binary data to check for error.

3. If there is an error, display the position of error

4. Else remove the redundant bits and convert the binary data to ascii and display the output.

PROGRAM:

SENDER:

```
data = input("Enter binary data of any length:")
if not data or any(bit not in "01" for bit in data):
    print("Enter valid binary data.")
    raise SystemExit

m=len(data)
r=0
while 2**r<m+r+1
    r+=1

n=m+r
h=[0]*(n+1)
j=0
for i in range(1,n+1):
    if i==(i-1):
        h[i]=int(data[j])
        j+=1

for i in range(r):
    p=2**i;
    for j in range(1,n+1):
        if j==p and j!=p
            h[p]^=h[j]
```

```
code = ''.join(str(h[i]) for i in range(1, n+1))

with open("channel", "w") as f:
    f.write(code)
print("Encoded data:", code)
print("Saved to channel file.")
```

RECEIVER

```
with open("channel", "r") as f:
    code = f.read().strip()

if not code or any(bit not in "01" for bit in
    code):
    print("Invalid channel data.")
    raise SystemExit.
h = [0] + [int(bit) for bit in code]
n = len(code)
error = 0

p = 1
while p <= n:
    parity = 0
    for i in range(1, n+1):
        if i & p:
            parity ^= h[i]
    if parity:
        error += p
    p *= 2
if error:
    if error <= n:
        print("Error at position:", error)
        h[error] ^= 1
        print("Corrected code:",
              ''.join(str(h[i]) for i in range(1, n+1)))
```

```
    else:
        print("Invalid error position.")
        raise SystemExit

else:
    print("NO error detected.")
    pr

data = ''
    for i in range(1,n+1):
        if i != (i-1):
            data += str(h[i])
print("Original data:", data).
```



Exp:-7.

FLOW CONTROL AT DATA LINK LAYER:

Aim:

Write a program to implement flow control at data link layer using SLIDING WINDOW PROTOCOL.

Simulate the flow of frames from one node to another.

Program should achieve at least below given requirements. You can make it a bidirectional program wherein receiver is sending its data frames with acknowledgement (Piggybacked).

Create a sender program with following features.

1. Input window size from the user.
2. Input a Text message from the user.
3. Consider 1 chapter per frame.
4. Create a frame with following fields.
5. Send the frames.
6. Wait for the acknowledgement.
7. Reader a file called receiver. Buffer.
8. Check Ack field for Acknowledgement

FOR

A Create a receiver file.

1. Reads a file called Sender_Buffer
2. Check the frame no.
3. If the frame no. are as expected, write the appropriate Ack no. in the Receiver_Buffer file else write NACK no. in the Receiver_Buffer file.

PROGRAM:

SENDER.

```
import time
msg = input("Enter message:")
w = int(input("Enter window size:"))
start=0
while start < len(msg):
    batch = [f"{i+1}. {msg[i]}" for i in range
             (start, min(start+w, len(msg)))]
    with open("Sender_Buffer", "w") as f:
        f.write("\n".join(batch))
    print("Sent", batch)
    input("Run receiver.python press Enter, here")
    with open("Receiver_Buffer", "r") as f:
        ack = int(f.read())
    if ack == start + len(batch)+1:
        print("Ack received:", ack)
        start += len(batch)
    else:
        print("Nack received. Resending frames...")
        time.sleep(1)
```

```
print("Transmission complete!")
```

RECEIVER:

```
expected = 1

with open("Sender_Buffer", "r") as f:
    frames = f.readlines()

for frame in frames:
    no, data = frame.strip().split(",")
    no = int(no)

    if no == expected:
        print("Received frame :", no, data)
        expected += 1

    else:
        print("NAck:", expected)
        break

with open("Receiver_Buffer", "w") as f:
    f.write(str(expected))

print("Ack:", expected)
```

EX.NO: 8

Nmap Live host discovery using Tryhackme.

* Aim:

To discover live host on a network using Nmap host-discovery scans. ARP Scan, ICMP & TCP/UDP ping scan & to understand which protocol operates at which layer of the TCP/ model.

* Introduction:

* Nmap ("Network Mapper") is a free, open Source network-scanning tool created by Gordon Lyon.

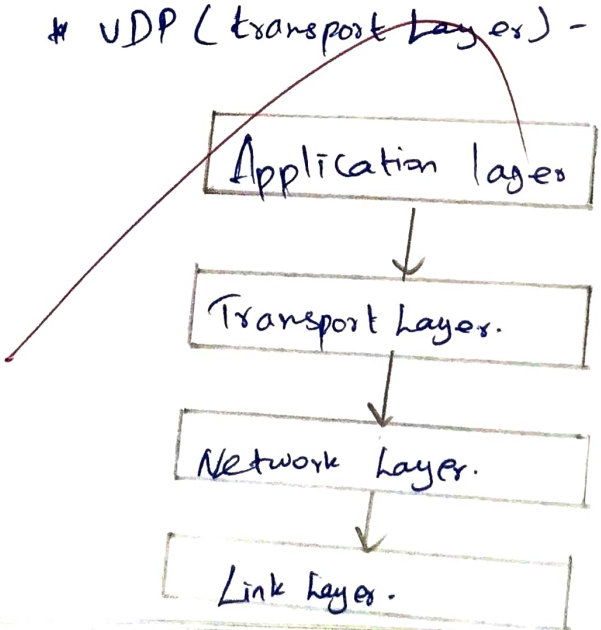
* Before Nmap can usefully scan ports on a target. It first needs to know whether the target is even online.

→ ARP (Link Layer) - for host on the same local Subnet

* ICMP (Network Layer) - Echo, timestamp.

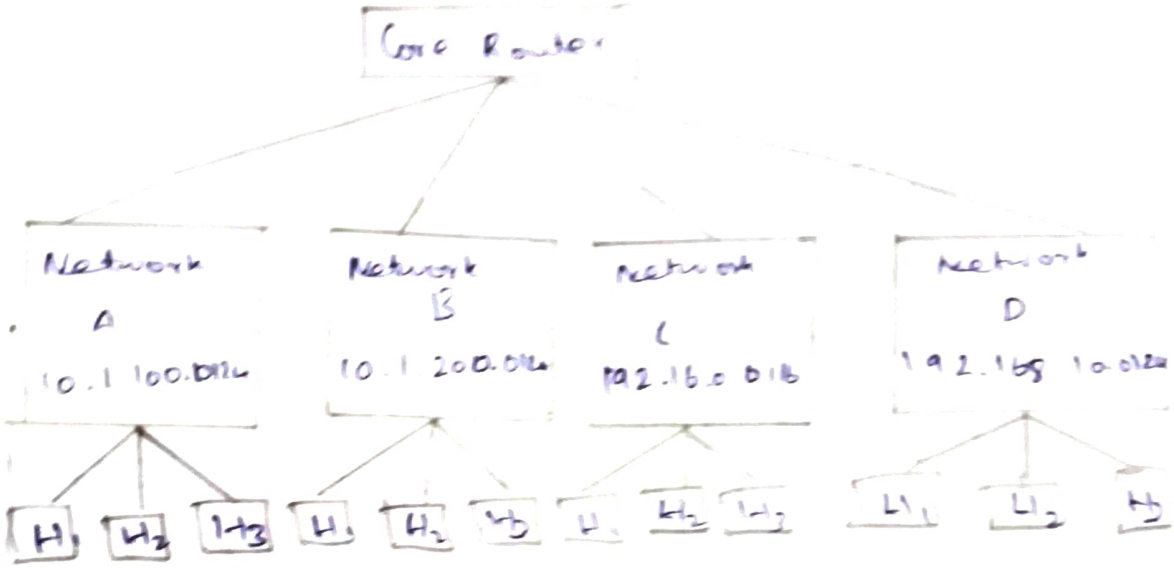
* TCP (Transport layer) - SYN + Ack ping probe

* UDP (transport ~~Layer~~) - relies on ICMP.



* Network Segments & Subnetworks

It is a group of computers connected by a shared physical medium. In short a Segment is a physical connection, a Subnet is a logical one



* Target Specification in Nmap

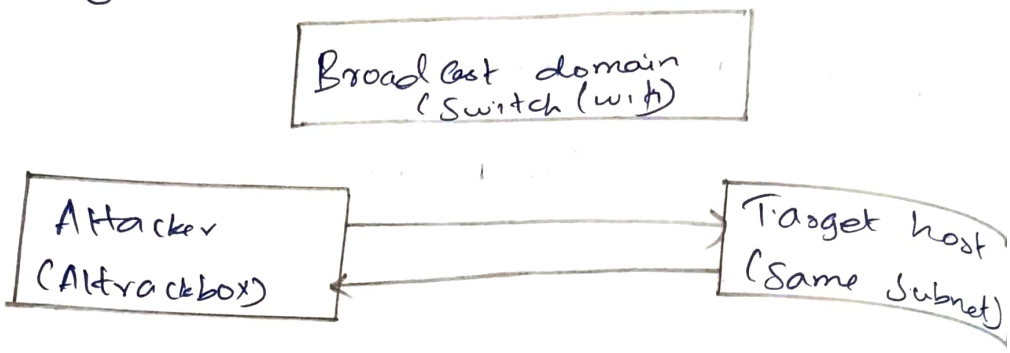
* A list nmap 10.10.15
* A range nmap 10 11.12.15.20
* A Subnet / CIDR : nmap 10.10.1.0/24
* A file of target : nmap -iL List-of-hosts.

* Lab Setup on TryHackme.

1. Open the room of log in to your account.
2. Read Task 1 & Task 2.
3. In task 4 click Start to deploy the target network
4. Click Start attack box to launch the browser
5. Once the attacker box has booted open a terminal inside it.
6. Note the Connection IP.

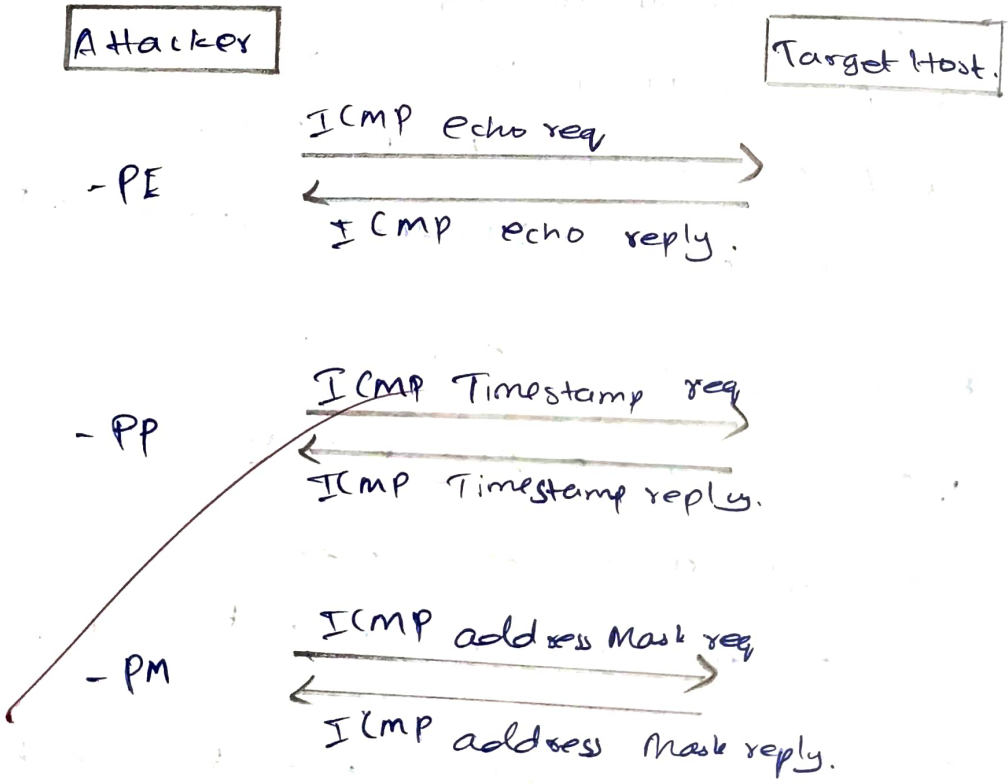
* ARP Host discovery.

On a local ethernet os with communication requires the destination MAC address. A reply proves the host is up



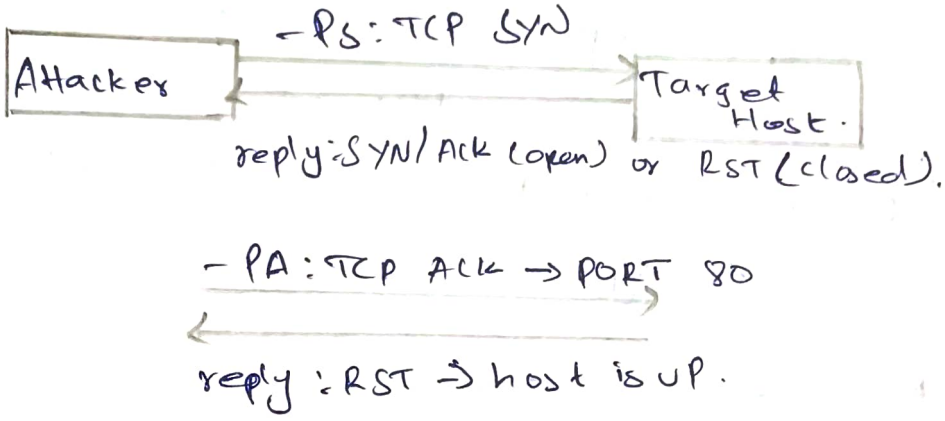
* ICMP Host discovery.

ICMP offers several request pairs that can reveal a live host. Windows system & many firewalls block ICMP echo by default. Remember that on your own subnet, an ARP exchange still happens.



* TCP & UDP ping discovery.

When ICMP is filtered, Nmap can send crafted TCP or UDP packets & infer that a host is alive purely from the fact that it responds at all - regardless of whether the probed port is open or closed.



* Reverse-DNS Lookup:

Reverse DNS (rDNS) resolves an IP address back to a hostname. Nmap performs lookups on live hosts by default; -R force a lookup on every target. Hostnames such as dict.domain.lo

EX.NO:9 Implementation of Subnetting in Cisco Packet Tracer.

* Aim:

To implement Subnetting in cisco packet tracer Simulator.

* Creating a Network Topology:

The first steps in implementing.

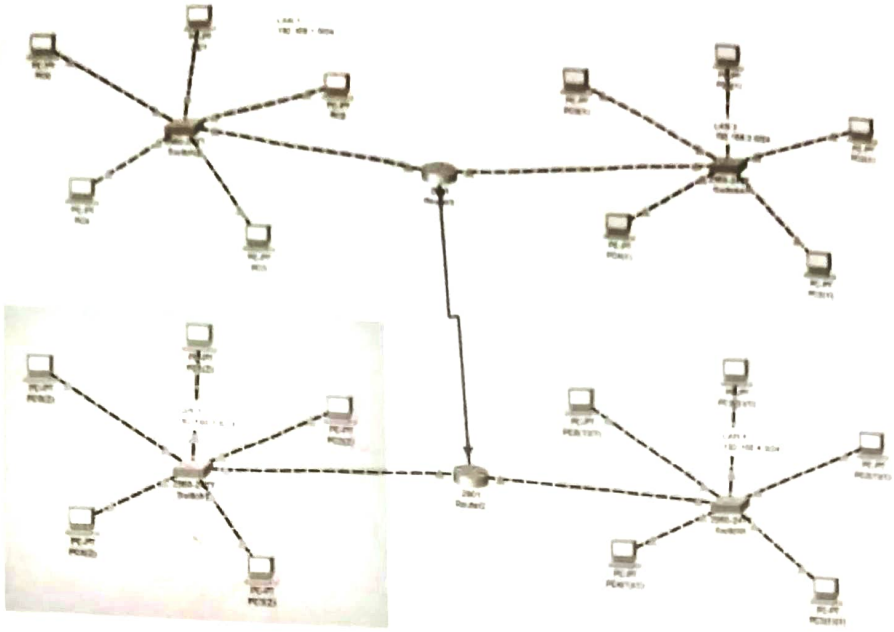
Classless IP subnetting is to create a network topology in packet tracer. This will create a blank network topology.

* Adding that devices.

Once we have created our network topology, we can add devices to it. Here, we will be adding routers, Switches & PCs. Then connect the devices by changing a cable from the device port to another devices port.

* Subnetting:

To Subnet the network address of 192.168.1.0/24 to provide enough space for at least 5 addresses for end devices, the Switch & the router. we can user a 127 Subnet mask.



=> The IP addressing for the network shown in the topology as follows:-

- Router R1:
- Gigabit Ethernet 0/0 : 192.168.1.1.
- Gigabit Ethernet 0/1 : 192.168.2.1
- Switch S1:
- Fast Ethernet 0/1 : 192.168.1.0/24.
- PC1 : 192.168.1.11
- PC2 : 192.168.1.12.
- PC3 : 192.168.1.13
- PC4 : 192.168.1.14
- PC5 : 192.168.1.15.
- Fast ethernet 0/2 : 192.168.2.0/27
- PC1 : 192.168.2.11
- PC2 : 192.168.2.12
- PC3 : 192.168.3.13
- PC4 : 192.168.4.14.
- PC5 : 192.168.5.15.

CONFIGURING THE DEVICES:

Now that we have added our devices and connected them, we can start configuring them. We will start by configuring the router. Right click on the router and select "CLI". This will open the command-line interface for the router. In the CLI, enter the following commands.

#enable
#configure terminal
#interface Fast Ethernet 0/0
#ip address {IP address} {Subnet mask}
#no shutdown
#exit.
interface Fast Ethernet 0/1
ip address {IP address} {Subnet mask}
no shut down
exit.

To configure the gigabit Ethernet interface on the router, you can follow these steps:

1. Right-click on the router and select "CLI".
2. Enter the following.

Commands: enable configure termined
interface Gigabit Ethernet 0/0
ip address {IP address} {Subnet mask} no shutdown
exit.

To Replace "{IP address}" and "{Subnet mask}" with your desired IP address and Subnet mask.

These Commands configure the Gigabit Ethernet interface with an IP address and subnet mask, and enable the interface.

TESTING THE NETWORK:

Now that our network is configured we can test the network. Open a Command prompt on each PC and try to ping the other PC. if the ping is successfull, then the network is functioning properly. We can also use the "ping" command to test Connectivity between the router and the PCs.

Fire	Last Status	Source	Destination	Type	Color	Time(sec)	Periodic	Num
●	Successful	PC4(2)	Router2	ICMP		0.000	N	12
●	Successful	PC4(2)	PC2(1)(1)	ICMP		0.000	N	13
●	Successful	PC0	Router0	ICMP		0.000	N	14

EXP:10. Internetworking with routers in CISCO Packet tracer.

Aim:

Interietworking with routers in CISCO PACKET TRACER Simulator

In this network, a router and 2 PCs are used Computers are connected with routers using a Copper straight-through cable. After forming the network, to check network connectivity a simple PDU is transferred from PC0 to PC1.

Procedure:

Step-1 (Configuring Router1):

1. Select the router and Open CLI.
2. Press ENTER to start configuring Router1.
3. Type enable to activate the privileged mode.

Router1 CLI:

Router>enable

Router#config t

Enter configuration commands, one per line. End with CNTL/2. Router(config)# interface FastEthernet 0/0

Router(config-if)#ip address 192.168.10.1 255.255.255.0 Router(config-if)#no shutdown

Router(config-if)#

Step 2 (Configuring PCs):

1. Assign IP Addresses to every PC in the network.

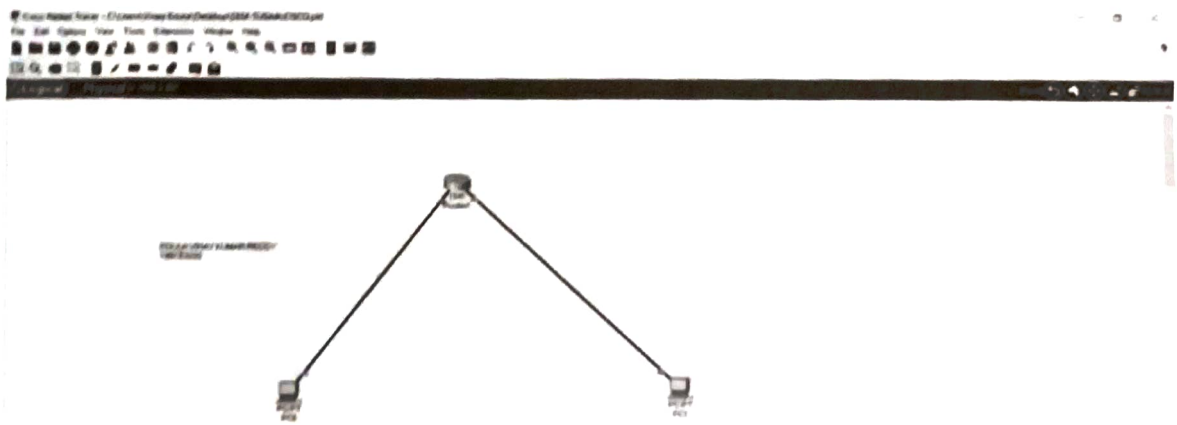
2. Select the PC, click to the desktop and select IP Configuration and assign an IP address, Default gateway Subnet Mask.

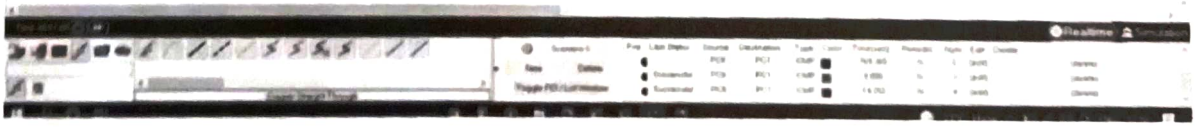
3. Assign the default gateway of PC1 as 192.168.2.1

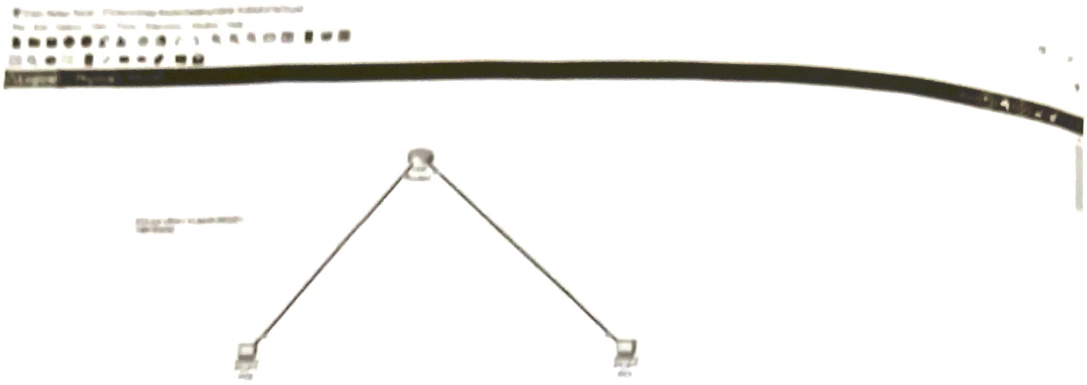
Step 3 Connecting PCs with Router

1. Connect FastEthernet 0 port of PC 0 with FastEthernet 0/0 port of Router1 using a Copper Straight through Cable.

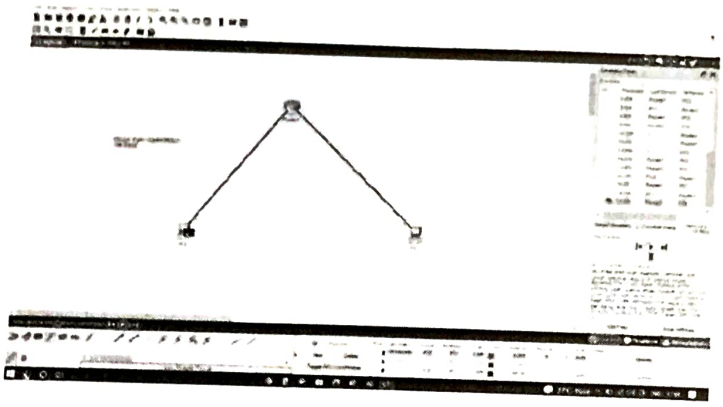
2. Connect FastEthernet 0 port of PC1 with FastEthernet port of Router1 using a Copper straight-through cable.

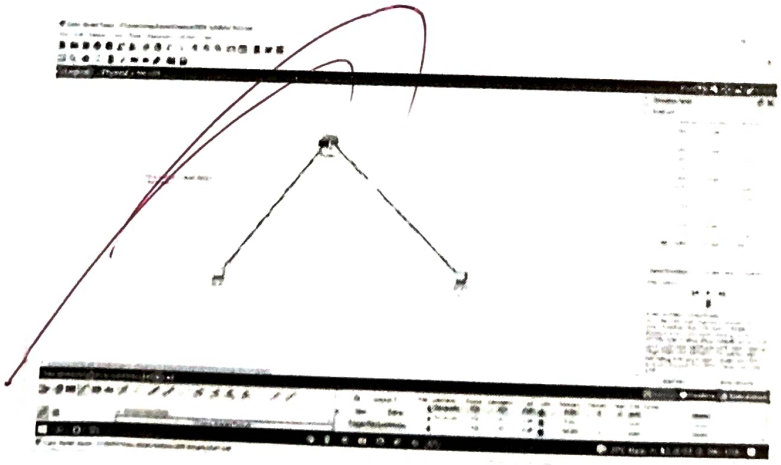












(10%) Aim b)

Design and Configure an internetwork using wireless router, DHCP server and internet cloud.

Part 1: Build a Simple Network in Logical Topology Workspace

Step1: Launch Packet Tracer.

Step 2: Build the topology

a) Add network devices to the workspace.

Using the device selection box, add the network devices to the workspace as shown in topology diagram.

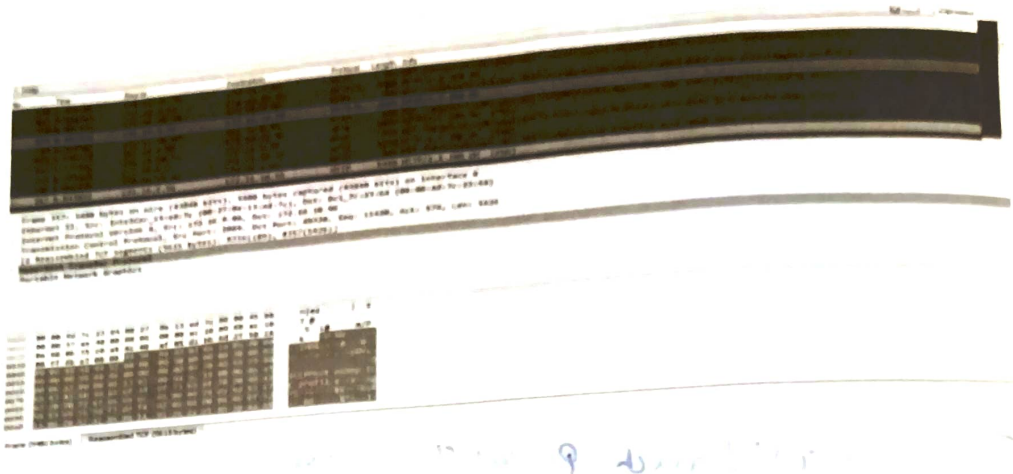
b) Change display names of the network devices.

c) Add the physical cabling between devices on the workspace.

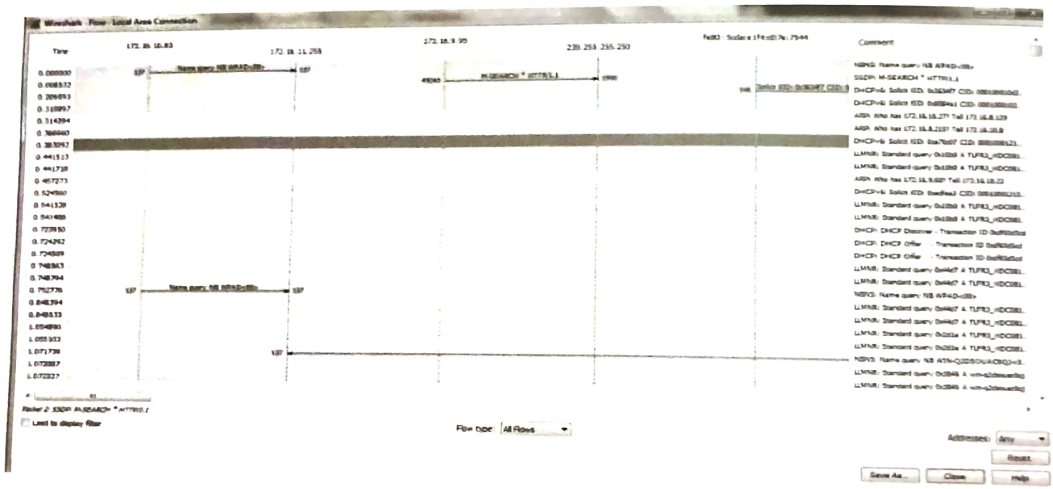
Part 2: Configure the Network Devices.

a. Create the wireless network on the wireless router.

In the DHCP Server Settings verify the Enabled button is selected and configure the Static IP address of the DNS Server as 208.67.220.220 as shown in the figure.



b) Click on the Save Settings tab.



Step 3: Configure the Board Card

a) Install software or cards if necessary

b) Identify the slots and the bus

c) Identify the type of boards

while (i) - the configuration file stored under INTERRUPT - the objects in the interrupt configuration window select the slot in the list of network or board, to type

Part 2 Verify Connectivity

a) Verify that the PC is running the configuration utility for card

b. Test Connectivity to the Cisco.com Server from the PC.

Ex.No

From the command prompt, issue the command ping Cisco.com. It may take a few seconds for the ping to return. Four replies should be received as shown in the figure.

EX.NO:11

Simulation of RIP using Cisco Packet tracer.

Aim:

To Simulate Routing information Protocol (RIP) using CISCO packet tracer and observation dynamic routing between different networks.

IP Configuration:

Device	Interface	IP Configuration
PC0	Fast ethernet	10.0.0.2/8
Router 0	Fa0/1	10.0.0.1/8
Router 0	Se0/1/1	192.168.1.254/30
Router 0	Se0/1/0	192.168.1.249/30
Router 1	Se0/1/0	192.168.1.250/30
Router 1	Se0/1/1	192.168.1.246/30
Router 2	Se0/1/0	192.168.1.243/30
Router 2	Se0/1/1	192.168.1.253/30
Router 2	Fa0/1	20.0.0.1/30
PC1	Fast ethernet	20.0.0.2/30

Procedure:

1. Create the topology with 3 routers and 4PCs.

2. Connect the routers using serial Cables and connect Pcs to Pc respective routers.

3. Assign IP Address to all Pc's and routers interface.

4. Enable the router interface.

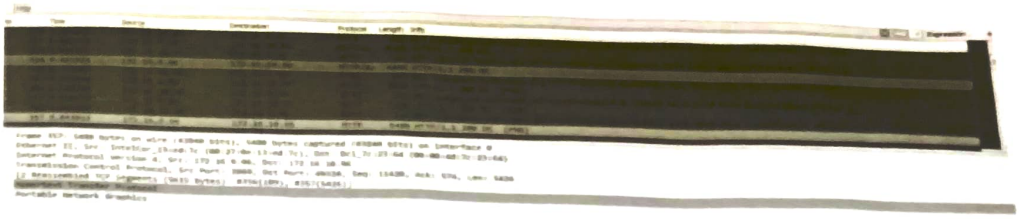
5. Configure the remaining interface accordly to the given IP table.

6. Enable RIP on Router 0 and configure RIP on Router 1 and 2.

7. From PC0, use ping 20.0.0.2 to verify Connectivity.

8) Use tracert to observe the route taken by packets.

9) Disconnect the direct Router 0- Router 2 Click and observe the RIP routes the traffic through Router 1.







Aim:

a) Implement a client server application using TCP socket

b) Implement a chat program using TCP socket programming

TCP

```
import socket
import threading

# ... SERVER ...

def start_server():
    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

    server.bind(('127.0.0.1', 5000))
    server.listen()
    print("[SERVER] listening on 127.0.0.1:5000")

    conn, addr = server.accept()
    print(f"[SERVER] connected by {addr}")

    while True:
        data = conn.recv(1024).decode()
        if not data or data.lower() == "quit":
            print("[SERVER] connection closed")
            break
        print(f"[SERVER] Received: {data}")
        conn.send("Server received: ".encode())
```

```
    Conn.close()
    Server.close()

    #.....CLIENT.....

    def start_client():
        client=socked.socket(socket.AF_INET,socked.
                                        -STREAM)

        client.connect(('127.00.1',5000))
        print("[CLIENT] connected to server")

    while True:
        msg=input("Enter message('quit' to exit):")

        client.send(msg.encode())
        if msg.lower()=='quit':
            break.
        response=client.recv(1024).decode()
        print(f"[SERVER RESPONSE]{response}")

    client.close()

#.....MAIN.....

if __name__=="__main__":

    # Run server in a backgroung thread.

    Server_thread=threading.Thread(target=start_
daemon=True)                                    server)

    server=thread.start()

    # Run client in main thread

    Start_client()
```

```
    conn.close()
server.close()

#.....CLIENT.....

def start_client():
    client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

    client.connect(('127.0.0.1', 5000))
    print("[CLIENT] Connected to server")

    while True:
        msg = input("Enter message (or 'quit' to exit): ")
        client.send(msg.encode())
        if msg.lower() == 'quit':
            break
        response = client.recv(1024).decode()
        print(f"[SERVER RESPONSE] {response}")

    client.close()

#.....MAIN.....

if __name__ == "__main__":
    # Run server in a background thread
    server_thread = threading.Thread(target=start_server, daemon=True)
    server_thread.start()

    # Run client in main thread
    start_client()
```

Output:

```
========== RESTART: C:/Users/tkala/Downloads/tcp.py ==========
[SERVER] Listening on 127.0.0.1:5000[CLIENT] Connected to server
[SERVER] Connected by ('127.0.0.1', 53458)Enter message (or 'quit' to exit):
hello
[SERVER] Received: hello
[SERVER RESPONSE] Server received: hello
Enter message (or 'quit' to exit):
========== RESTART: C:/Users/tkala/Downloads/tcp.py ==========
[SERVER] Listening on 127.0.0.1:5000[CLIENT] Connected to server
[SERVER] Connected by ('127.0.0.1', 60328)Enter message (or 'quit' to exit):
Hi
[SERVER] Received: Hi
[SERVER RESPONSE] Server received: Hi
Enter message (or 'quit' to exit): quit
[SERVER] Connection closed by client
```

UDP

```
import socket
import threading

#.... SERVER ....

def udp_server():
    s=socket.socket(socket.AF_INET, sockt.SOCK_DIAGRAM)

    s.bind(("127.0.0.1", 12000))
    print("[SERVER] Running on 127.0.0.1:12000")

    while True:
        data, addr = s.recvfrom(1024)
        msg=data.decode()
        if msg.lower()=='quit':
            print(f"[SERVER] client {addr} ended the
            Connection.")
            break
```

```
        print(f"[SERVER] Received from {addr}: {msg}")
        s.sendto(data, addr)  # Echo back
    s.close()
    print("[SERVER] Stopped.")

#.... CLIENT ....

def udp_client():
    c = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

    server = ('127.0.0.1', 12000)

    while True:
        msg = input("Enter message (or 'quit' to stop): ")
        c.sendto(msg.encode(), server)
        if msg.lower() == 'quit':
            print("[CLIENT] Closing connection...") break

        data, _ = c.recvfrom(1024)
        print("Echo from server:", data.decode())

    c.close()
    print("[CLIENT] Stopped.")
```

```
#.... MAIN ....
if __name__ == "__main__":
    threading.Thread(target=udp_server,
                     daemon=True).
    start()
    udp_client()
```

OUTPUT:

```
Enter message (or 'quit' to stop): [SERVER] Running on 127.0.0.1:12000hi

[SERVER] Received from ('127.0.0.1', 49670): hi
Echo from Server: hi
Enter message (or 'quit' to stop): hello
[SERVER] Received from ('127.0.0.1', 49670): hello
Echo from Server: hello
Enter message (or 'quit' to stop): how are you
[SERVER] Received from ('127.0.0.1', 49670): how are you
Echo from Server: how are you
Enter message (or 'quit' to stop): quit
[CLIENT] Closing connection...[SERVER] Client ('127.0.0.1', 49670) ended the connection.

[CLIENT] Stopped [SERVER] Stopped
```

EXP.no:13 IMPLEMENT YOUR OWN PING PROGRAM.

```
import os
import platform
import subprocess

def ping(host = "google.com", count=4):
    param = "-n" if platform.system().lower()=="windows"
                                    else "-c"

    command = ["ping", param, str(count), host]

try:
    output = subprocess.check_output(command, universal_
                                    newlines=True)

    print(output)
    except Exception as e:
        print("Ping failed :(", e)
if __name__ == "__main__":
    ping("8.8.8.8")
```

Output

```
Pinging 8.8.8.8 with 32 bytes of data:
Reply from 8.8.8.8: bytes=32 time=6ms TTL=119
Reply from 8.8.8.8: bytes=32 time=9ms TTL=119
Reply from 8.8.8.8: bytes=32 time=5ms TTL=119
Reply from 8.8.8.8: bytes=32 time=9ms TTL=119

Ping statistics for 8.8.8.8:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 5ms, Maximum = 9ms, Average = 7ms
```

Aim:

To write a code using RAW sockets to implement packet sniffing.

```
from scapy.all import sniff
from scapy.layers.inet import IP, TCP, UDP, ICMP.


def packet_callback(packet):

    if IP in packet:

        ip_layer = packet[IP]
        protocol = ip_layer.proto
        src_ip = ip_layer.src
        dst_ip = ip_layer.dst

    # Determine the protocol.

        protocol_name = ""

        if protocol == 1:
            protold_name = "ICMP"

        elif protocol == 6:
            protocol_name = "TCP"

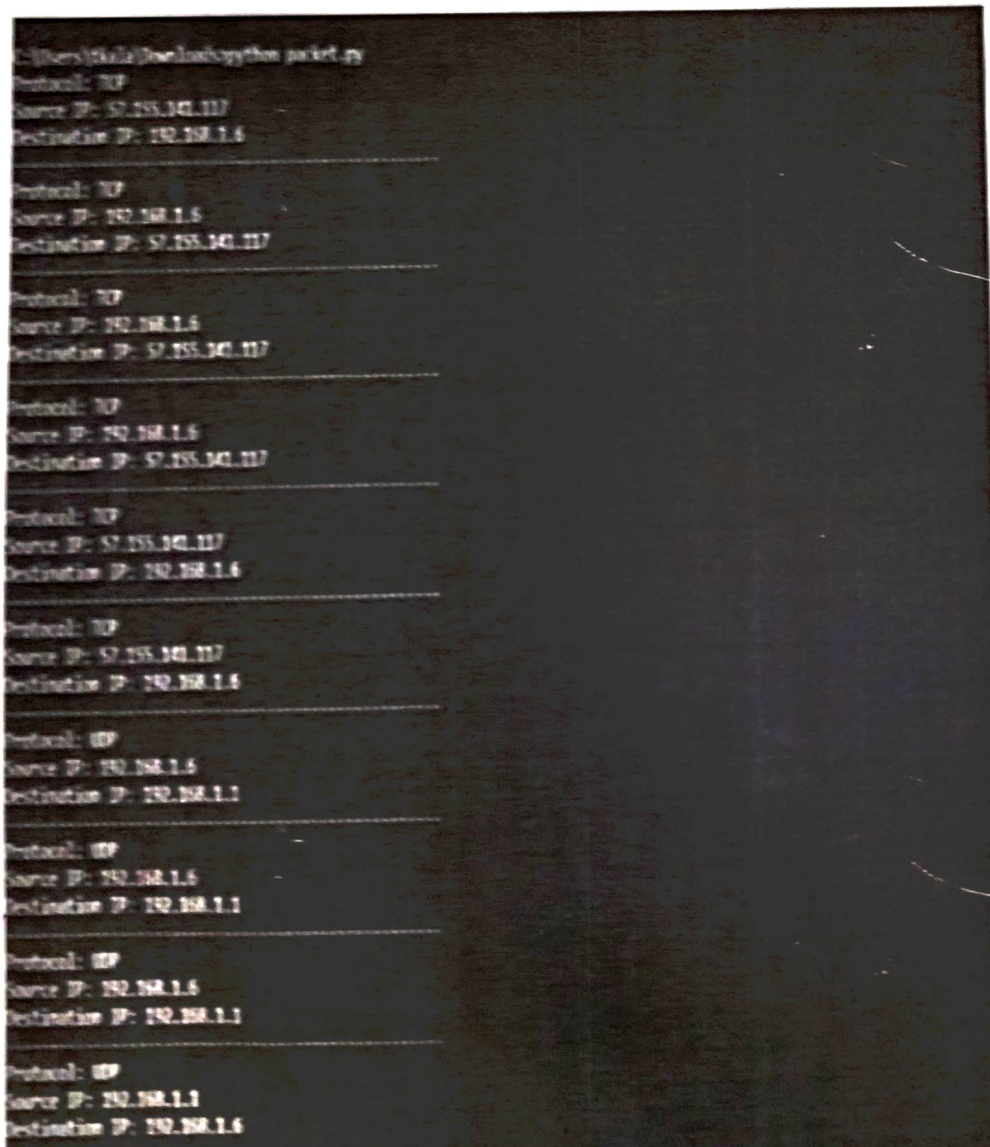
        elif protocol == 17:
            protocol_name = "UDP"

        else:
            protocol_name = "Unknown protocol"
```

```
# Print packet details
    print(f"Protocol: {protocol_name}")
    print(f"Source IP: {src_ip}")
    print(f"Destination IP: {dst_ip}")
    print("-" * 50)

# Capture packets on the default network interface.
sniff(iface='Wi-Fi', prn=packet_callback, filter="ip",
                                          store=0)
```

OUTPUT:



Completed
13/10/26